

83. You are given an undirected graph represented by a list of edges and the number of vertices n . Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] and $n = 4$

```
def hamiltonian(graph, path, pos, n):
    if pos == n:
        return graph[path[pos-1]][path[0]] == 1

    for v in range(1,n):
        if graph[path[pos-1]][v] and v not in path:
            path[pos] = v

            if hamiltonian(graph,path,pos+1,n):
                return True

            path[pos] = -1

    return False

n = 4
edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]

graph = [[0]*n for _ in range(n)]

for u,v in edges:
    graph[u][v] = graph[v][u] = 1

path = [-1]*n
path[0] = 0

if hamiltonian(graph,path,1,n):
    print(path+[0])
else:
    print(False)

... [0, 1, 2, 3, 0]
```

82. You are given an undirected graph represented by a list of edges and the number of vertices n . Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: Given edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)] and $n = 5$

```
def hamiltonian(graph, path, pos, n):  
  
    if pos == n:  
        return graph[path[pos-1]][path[0]] == 1  
  
    for v in range(1, n):  
  
        if graph[path[pos-1]][v] == 1 and v not in path:  
  
            path[pos] = v  
  
            if hamiltonian(graph, path, pos+1, n):  
                return True  
  
            path[pos] = -1  
  
    return False
```

```
n = 5  
  
edges = [(0,1),(1,2),(2,3),(3,0),  
         (0,2),(2,4),(4,0)]  
  
graph = [[0]*n for _ in range(n)]  
  
for u,v in edges:  
    graph[u][v] = graph[v][u] = 1  
  
path = [-1]*n  
path[0] = 0  
  
if hamiltonian(graph,path,1,n):  
    print(path+path[0])  
else:  
    print(False)
```

False

81. You and your friends are tasked with coloring a map using a limited set of colors, with the following rules: At each step, you can choose any region of the map that hasn't been colored yet and color it with any available color. Your friend Alice will then color the next region using the same strategy, followed by your friend Bob. You aim to maximize the number of regions you color. Given a map represented as a list of regions and their adjacency relationships, write a function to determine the maximum number of regions you can color. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4, k = 3

```
def graphColoring(graph, m, color, v, n):

    if v == n:
        return True

    for c in range(1, m+1):

        safe = True

        for i in range(n):
            if graph[v][i] and color[i] == c:
                safe = False

        if safe:
            color[v] = c

            if graphColoring(graph, m, color, v+1, n):
                return True

        color[v] = 0

    return False

n = 4
m = 3

edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]

graph = [[0]*n for _ in range(n)]

for u,v in edges:
    graph[u][v] = graph[v][u] = 1

color = [0]*n

if graphColoring(graph,m,color,0,n):
    print(color)
else:
    print("No Solution")
```

[1, 2, 3, 2]

CHAPTER-7

80. You and your friends are assigned the task of coloring a map with a limited number of colors. The map is represented as a list of regions and their adjacency relationships. The rules are as follows: At each step, you can choose any uncolored region and color it with any available color. Your friend Alice follows the same strategy immediately after you, and then your friend Bob follows suit. You want to maximize the number of regions you personally color. Write a function that takes the map's adjacency list representation and returns the maximum number of regions you can color before all regions are colored. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4

```
def graphColoring(n, edges):
    graph = [[0]*n for _ in range(n)]

    for u, v in edges:
        graph[u][v] = graph[v][u] = 1

    color = [0]*n

    def isSafe(v, c):
        for i in range(n):
            if graph[v][i] and color[i] == c:
                return False
        return True

    def solve(v):
        if v == n:
            return True

        for c in range(1, n+1):
            if isSafe(v, c):
                color[v] = c

                if solve(v+1):
                    return True

                color[v] = 0

        return False

    solve(0)

    print("Color Assignment:", color)
    print("Minimum Colors =", max(color))

n = 4
edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]

graphColoring(n, edges)

Color Assignment: [1, 2, 3, 2]
Minimum Colors = 3
```